Министерство образования Республики Беларусь

Учреждение образования

«Брестский государственный технический университет»

Кафедра ИИТ

Лабораторная работа №1

За седьмой семестр

По дисциплине: «Современные методы защиты информации»

Тема: «Исследование уязвимостей веб-приложений и программных интерфейсов, настройка механизмов аутентификации»

Выполнил:
Студент 3 курса
Группы ИИ-26(1)
Ерошин Е.А.
Проверила:
Хацевич А.С.

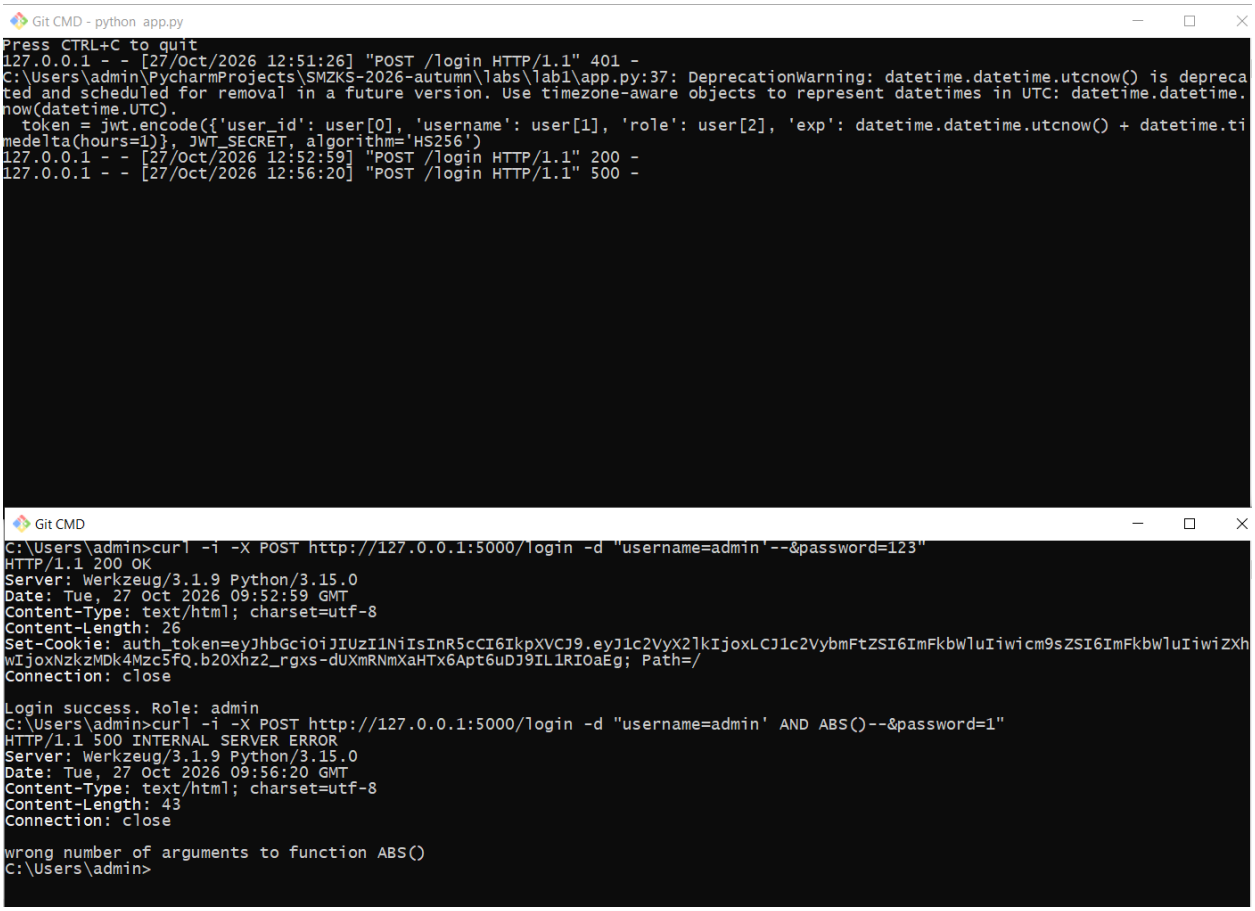
Брест 2026

Вариант 4

Вариант	Условие SQLi (Payload type)	Вектор XSS (Обязательный тег)	BOLA: ID жертвы	JWT: Целевая роль (role)	Защита: Директива CSP
1	Boolean-based (OR)	<script>	ID = 1 (admin)	admin	default-src
2	Tautology ('=')	<img> с обработчиком onerror	ID = 3	superuser	script-src с использованием nonce
3	Union-based (если применимо) или OR	<svg> с обработчиком onload	ID = 4	root	script-src с использованием hash
4	Error-based (вызов ошибки БД)	<body> с onload	ID = 5	administrator	strict-dynamic

Задание 1. Обход аутентификации через SQL-инъекцию и ее устранение

1. **Выполните эксплуатацию уязвимости SQL-инъекции через эндпоинт http://127.0.0.1:5000/login с помощью консольной утилиты curl для обхода аутентификации и получения сессионного токена администратора (токен понадобится в задании 4).**
2. **Устраните уязвимости SQL-инъекции путем переключения запроса на параметризованный вид в работающем экземпляре приложения через консоль Python.**
3. **Перезапустите сервер, повторите атаку пункта 1 и убедитесь, что сервер возвращает ошибку Invalid credentials (код 401).**



использовал ошибку некорректного вызова встроенной функции.

1. В SQL функция ABS(x) (модуль числа) обязательно требует один аргумент (например, ABS(-5)).
2. передали ABS() пустую, без аргументов.
3. База данных SQLite не смогла выполнить такой запрос и выдала ошибку синтаксиса/вызова.
4. Так как в коде в сервера стоит обработчик except Exception as e: return str(e), 500, эта внутренняя ошибка базы данных «вывалилась» прямо в ответ пользователю.

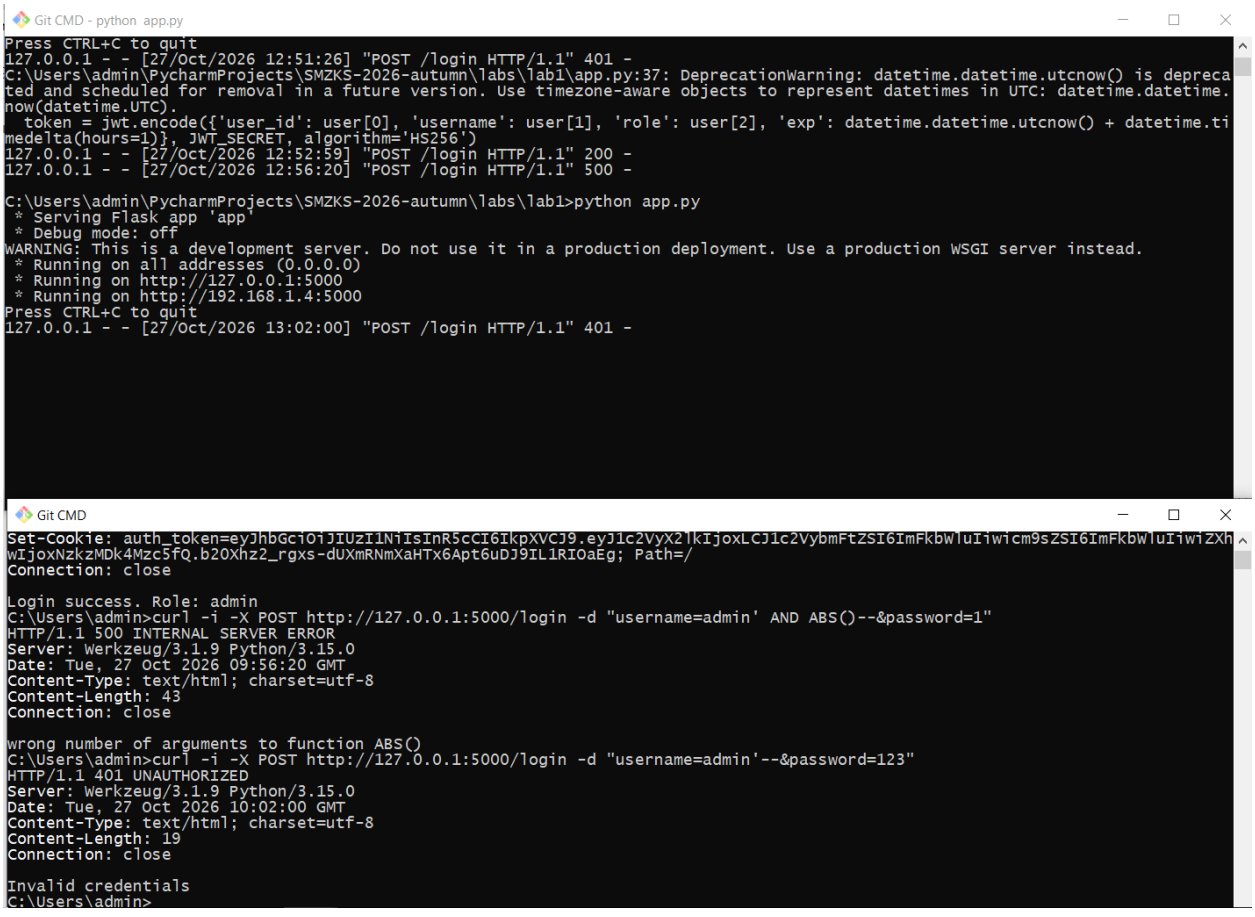
2.

```
# Исправленная функция /login
@app.route('/login', methods=['POST'])
def login():
    username = request.form.get('username', '')
    password = request.form.get('password', '')
    conn = sqlite3.connect('lab.db')
    c = conn.cursor()

    # БЕЗОПАСНЫЙ ЗАПРОС (параметризованный)
    query = "SELECT id, username, role FROM users WHERE username=? AND password=?"
```

```
    try:
        c.execute(query, (username, password))
        user = c.fetchone()
        if user:
            token = jwt.encode({'user_id': user[0], 'username': user[1], 'role': user[2], 'exp':
datetime.datetime.utcnow() + datetime.timedelta(hours=1)}, JWT_SECRET, algorithm='HS256')
            resp = make_response(f"Login success. Role: {user[2]}")
            resp.set_cookie('auth_token', token)
            return resp
        return "Invalid credentials", 401
    except Exception as e:
        return str(e), 500
```

3.

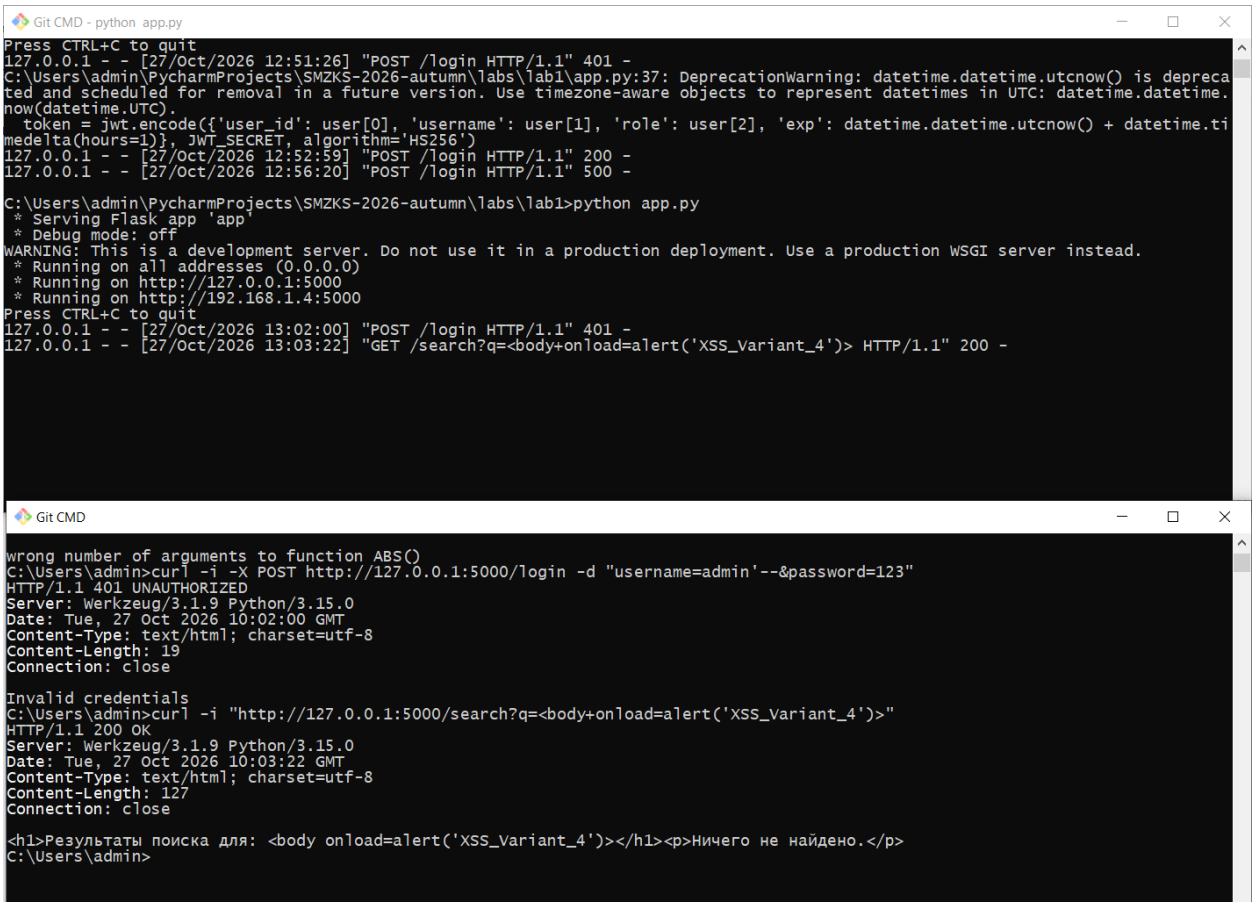


Задание 2. Эксплуатация межсайтового скриптинга (XSS)

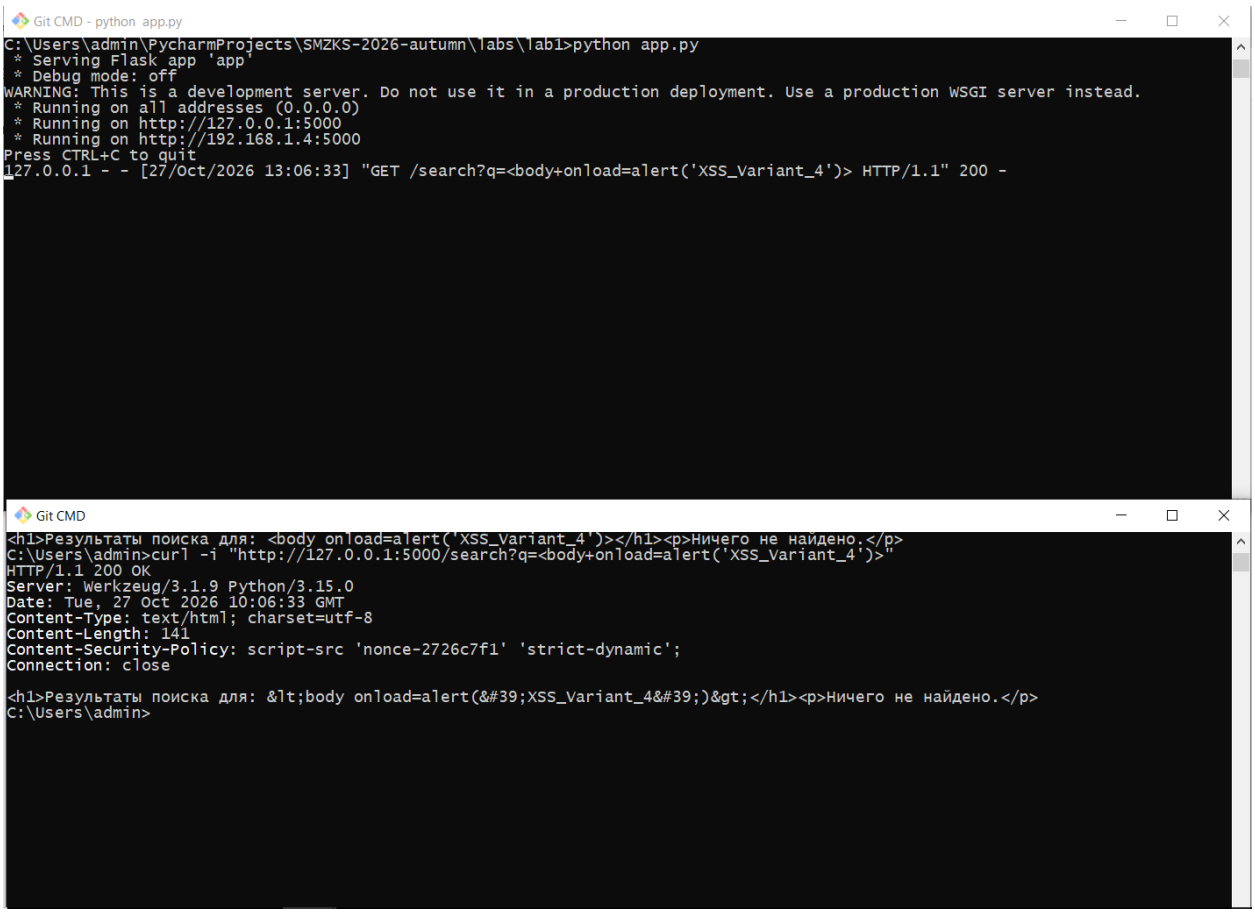
1. **Выполните отправку полезной нагрузки отраженного межсайтового скриптинга (Reflected XSS) через GET-запрос с помощью curl на эндпоинт http://127.0.0.1:5000/search?q= для демонстрации выполнения произвольного кода в контексте браузера.**

2. **Выполните внедрение защитного HTTP-заголовка Content-Security-Policy (CSP) в конфигурацию ответов сервера для блокировки выполнения несанкционированных сценариев.**

1.



2.




```
from markupsafe import escape


@app.route('/search', methods=['GET'])
def search():
    query = request.args.get('q', '')
    # 1. Экранирование ввода (превращает < в &lt;)
    safe_query = escape(query)

    template = f"<h1>Результаты поиска для: {safe_query}</h1><p>Ничего не найдено.</p>"
    resp = make_response(render_template_string(template))

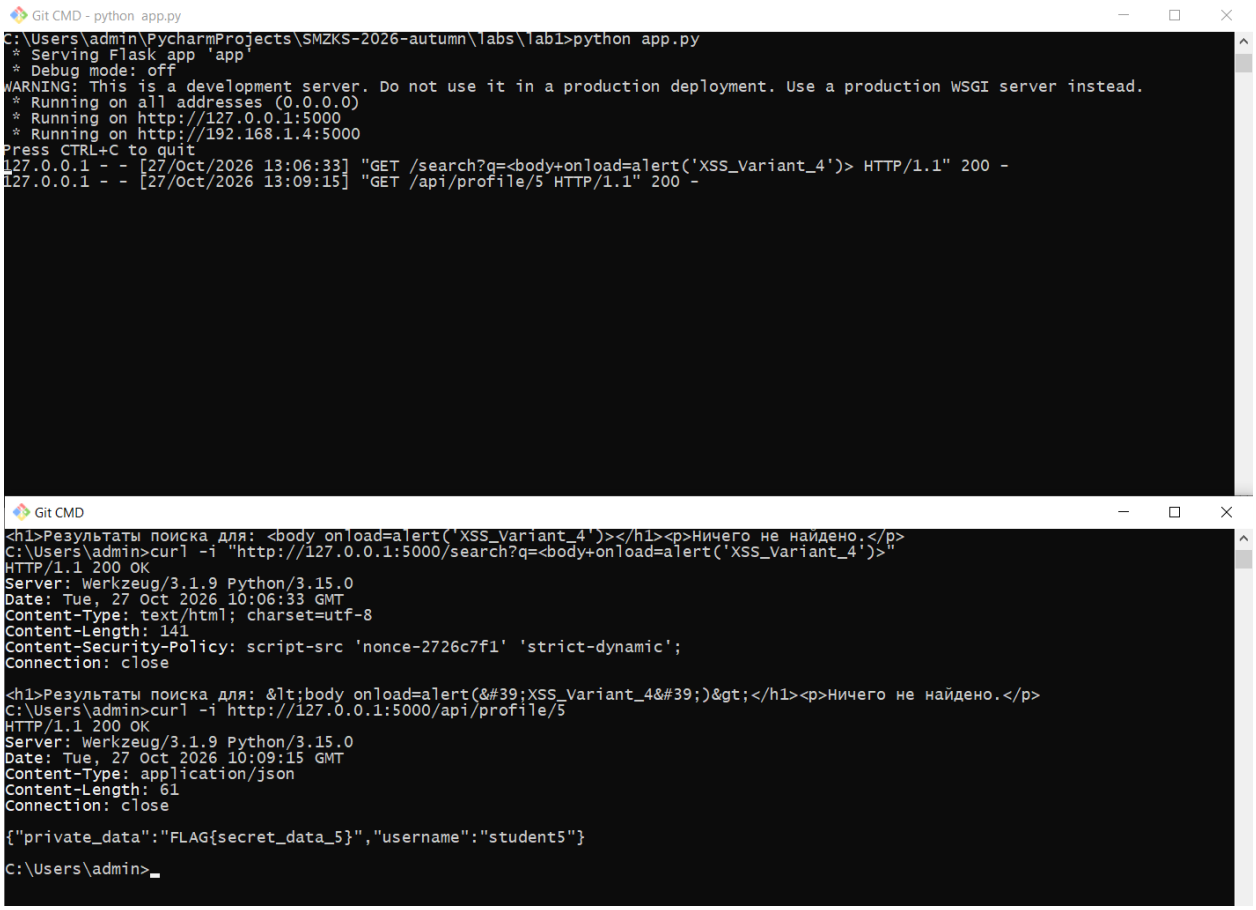
    resp.headers['Content-Security-Policy'] = "script-src 'nonce-2726c7f1' 'strict-dynamic';"

    return resp
```

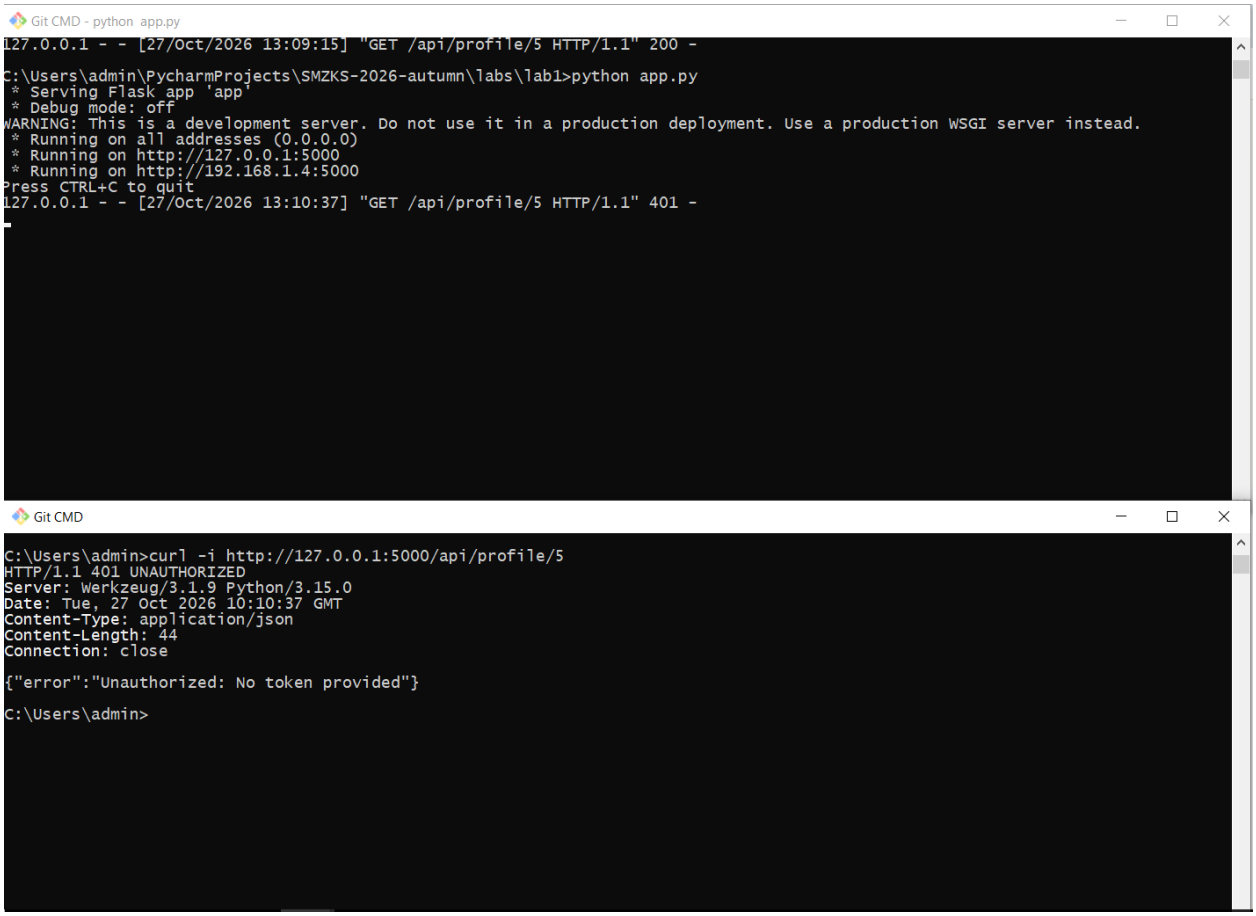
Задание 3. Нарушение разграничения доступа на уровне объектов (BOLA/IDOR)

1. **Выполните несанкционированное извлечение конфиденциальных данных администратора (Флаг системы) по прямому идентификатору через эндпоинт http://127.0.0.1:5000/api/profile/1 с помощью curl.**
2. **Выполните добавление проверки авторизации (валидации совпадения идентификатора сессии пользователя с запрашиваемым ID) в программную логику API.**

1.



2.




```
@app.route('/api/profile/<int:user_id>', methods=['GET'])
def get_profile(user_id):

    token = request.cookies.get('auth_token')
    if not token:
        return jsonify({"error": "Unauthorized: No token provided"}), 401

    try:
            decoded = jwt.decode(token, JWT_SECRET, algorithms=['HS256'])

        if decoded.get('user_id') != user_id and decoded.get('role') not in ['admin',
'administrator']:
            return jsonify({"error": "Forbidden: You cannot access other users' data"}), 403

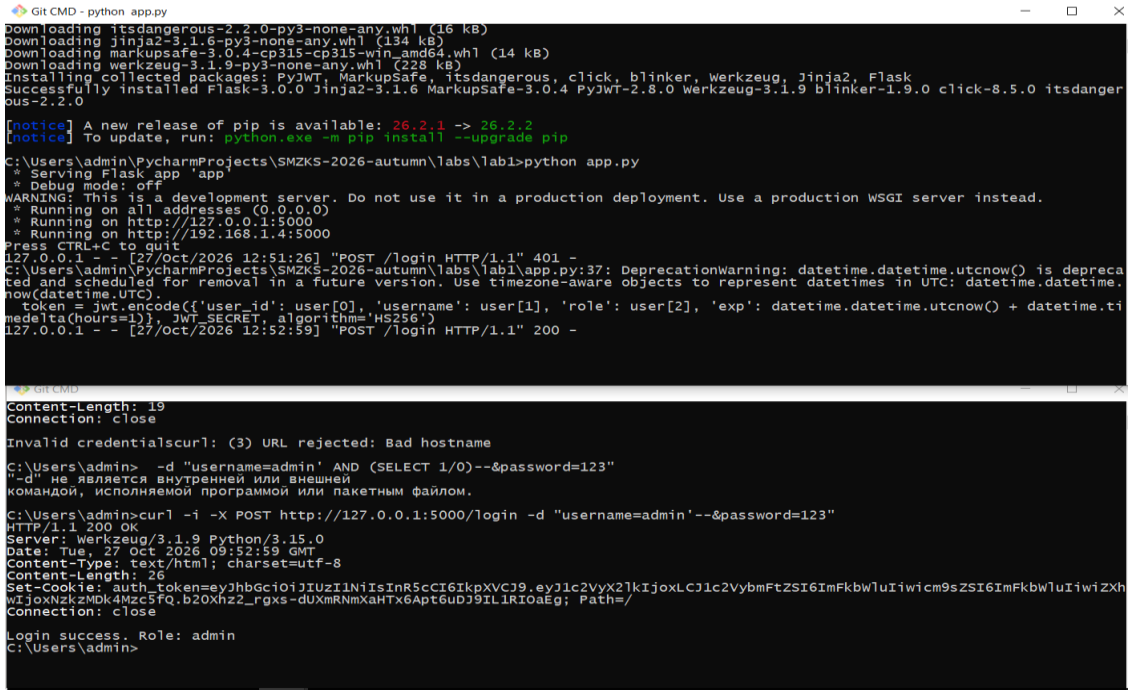
    except jwt.ExpiredSignatureError:
        return jsonify({"error": "Token expired"}), 401
```

```
except jwt.InvalidTokenError:
    return jsonify({"error": "Invalid token"}), 401


# Если проверка прошла, достаем данные из базы
conn = sqlite3.connect('lab.db')
c = conn.cursor()
c.execute("SELECT username, private_data FROM users WHERE id=?", (user_id,))
user = c.fetchone()
if user:
    return jsonify({"username": user[0], "private_data": user[1]})
return jsonify({"error": "User not found"}), 404
```

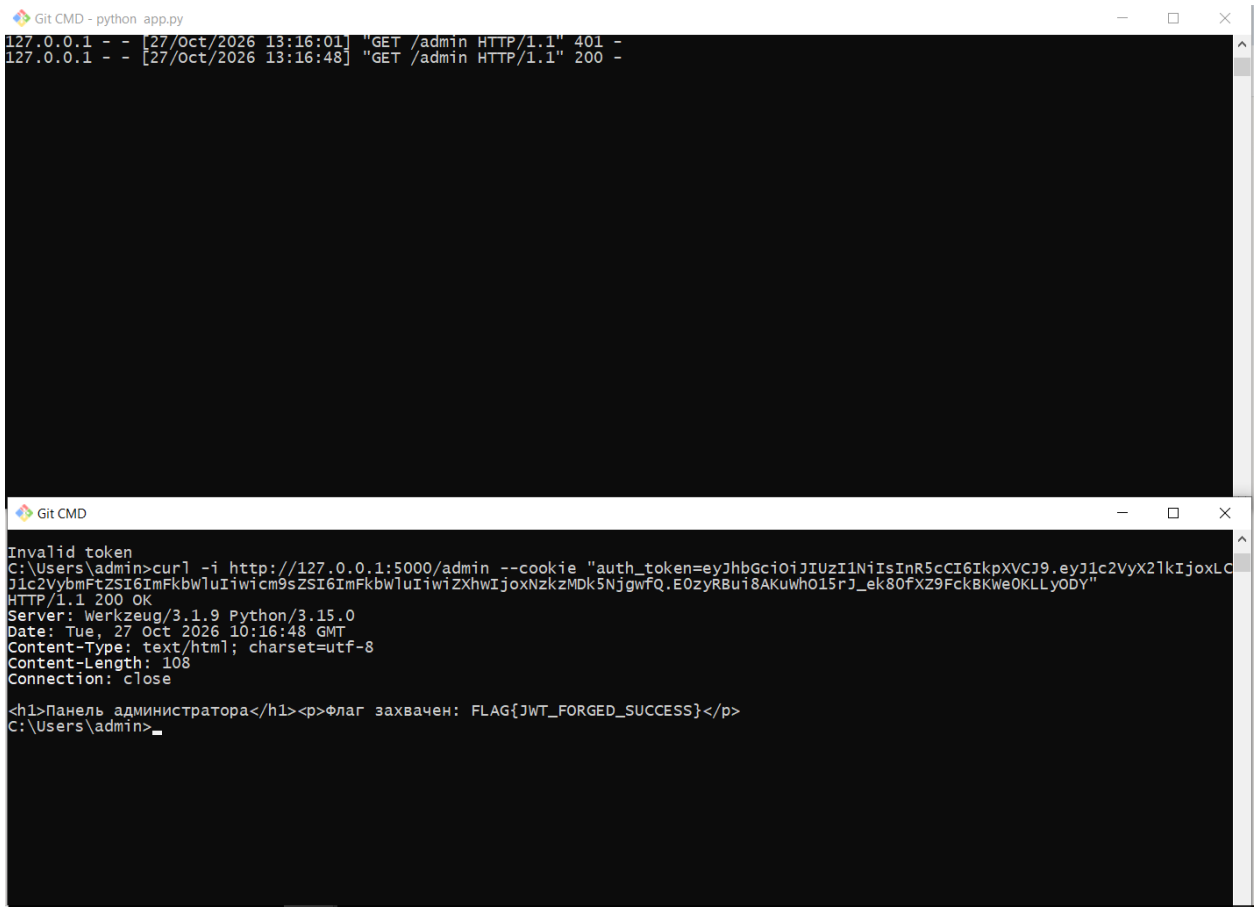
Задание 4 (JWT Forgery):

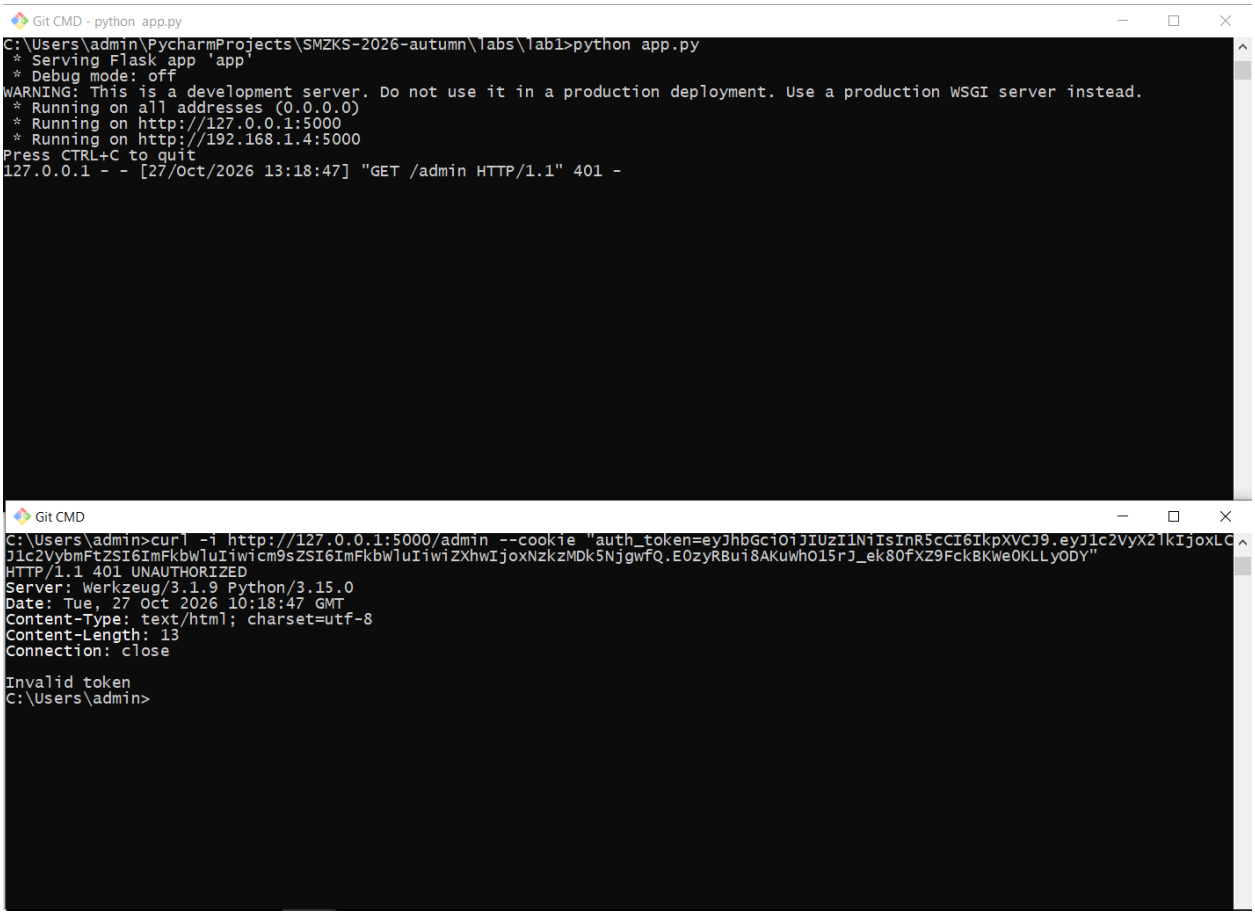
1. **Исходный слабый токен и результат его перебора (найденный секретный ключ).**
2. **Фрагмент консольного скрипта на Python, использованного для подписи поддельного токена с целевой ролью из индивидуального варианта.**
3. **Команда curl с подставленной cookies auth_token, демонстрирующая успешный доступ к закрытой панели администратора /admin (с выводом флага системы).**
4. **Фрагмент кода на Python, демонстрирующий перевод генерации токена на криптостойкий секрет (256 бит).**



```
```

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjoxLCJ1c2VybmFtZSI6ImFkbWluIiwicm9sZSI6ImFkbWluIiwiZXhwIjoxNzkzMDk4Mzc5fQ.b20Xhz2_rgxs-dUXmRNmXaHTx6Apt6uDJ9IL1RIOaEg





import secrets

Вместо "student2026" используем криптостойкий ключ 256 бит (32 байта)

JWT_SECRET = secrets.token_hex(32)

Задание 5 (Проектирование защитных механизмов):

1. **Развернутый ответ на пункт 1: корректно сформированный вызов метода set_cookie с обязательным указанием трех флагов (HttpOnly, Secure, SameSite) и академическим обоснованием их назначения.**

2. **Развернутый ответ на пункт 2: спецификация HTTP-заголовка Content-Security-Policy, соответствующая индивидуальной директиве варианта, с пояснением принципа ее работы против внедренных сценариев.**

   1. resp.set_cookie('auth_token', token, httponly=True, secure=True, samesite='Lax')
   Флаг : HttpOnly
   Назначение: Защита от XSS — запрещает доступ к cookie через JavaScript (document.cookie). Даже если злоумышленник внедрит скрипт, он не сможет украсть сессионный токен.
   Флаг : Secure
   Назначение : Передача cookie только по защищённому протоколу HTTPS. Защита от перехвата трафика (MitM-атак).
   Флаг : SameSite=Lax

Назначение : Защита от CSRF — cookie отправляется только при переходе по ссылке с того же сайта, но не при кросс-доменных запросах (например, при отправке формы с другого сайта).

2.
response.headers['Content-Security-Policy'] = "default-src 'self'; script-src 'strict-dynamic' 'nonce-123456'"
default-src 'self' — разрешает загрузку ресурсов только с того же домена.
script-src 'strict-dynamic' — разрешает выполнение скриптов, которые были загружены с доверенного источника (с неблокирующим nonce или хешем).
nonce-123456 — одноразовый токен, который должен совпадать с nonce в теге <script>. Это предотвращает выполнение произвольных скриптов.